

DOCUMENTACIÓN TÉCNICA

Sistema Experto: Recomendador de Herramienta de Dibujo Digital

Inteligencia Artificial II - Proyecto Final

Fundación Universitaria Los Libertadores

Carlos Andres Losada Diaz

2026

1. Introducción

1.1 Descripción del dominio

El sistema experto desarrollado tiene como dominio la recomendación de herramientas de dibujo digital. Dado un conjunto de características observables del usuario (dispositivo disponible, nivel de experiencia, tipo de arte que desea realizar y restricciones de presupuesto), el sistema aplica reglas de conocimiento experto para recomendar la herramienta más adecuada.

Las herramientas que el sistema puede recomendar son: Procreate, Clip Studio Paint, Krita, Adobe Photoshop, Adobe Illustrator, Inkscape, Affinity Designer, Figma y Autodesk Sketchbook.

1.2 Justificación y relevancia

La elección de una herramienta de dibujo digital es una decisión que confunde frecuentemente a artistas en formación. Existe una gran variedad de opciones con diferencias significativas en precio, plataforma, funcionalidades y curva de aprendizaje. Una mala elección puede resultar en inversión económica innecesaria, frustración y abandono del aprendizaje.

Este sistema permite incluir el concepto de artistas digitales experimentados, haciendo accesible una recomendación fundamentada a cualquier usuario que ingrese sus características de perfil.

1.3 Alcance del sistema

El sistema cubre:

- Usuarios con iPad, Tablet Android, PC Windows o Mac
- Perfiles de principiante, intermedio y profesional
- Tipos de arte: ilustración, vectorial, concept art, manga/comic, UI/UX y animación
- Restricciones de presupuesto y necesidades de colaboración

El sistema NO cubre:

- Recomendación de frameworks o plugins específicos dentro de cada herramienta
- Hardware de captura (scanners, cámaras)
- Herramientas de modelado 3D o edición de video
- Restricciones de idioma o soporte técnico regional

2. Diseño del sistema

2.1 Arquitectura general

El sistema sigue una arquitectura de tres capas desacopladas:

CAPA DE PRESENTACION	CAPA DE LOGICA	CAPA DE CONOCIMIENTO
app.py Interfaz web Streamlit - Formulario de entrada - Visualización resultados - Sección de explicación	motor_inferencia.py Motor de inferencia - encadenamiento_adelante() - explicar razonamiento() - porque()	base_conocimiento.py Base de conocimiento - 16 predicados - 15 reglas de producción - Mapeo de herramientas

2.2 Flujo de datos

El flujo de una consulta al sistema sigue los siguientes pasos:

1. El usuario selecciona sus características en el formulario web (app.py).
2. app.py construye un diccionario de hechos iniciales y llama a encadenamiento_adelante().
3. El motor de inferencia recorre las reglas de base_conocimiento.py iterativamente.
4. Por cada regla cuyas condiciones se satisfacen, agrega la conclusión a la memoria de trabajo.
5. El proceso se repite hasta alcanzar el punto fijo (ninguna regla nueva se dispara).
6. app.py recibe los hechos finales y filtra las recomendaciones para mostrarlas al usuario.
7. El usuario puede consultar la explicación del razonamiento o preguntar 'Por que?' sobre cada recomendación.

3. Base de conocimiento

3.1 Predicados del dominio

Todos los predicados son unitarios, definidos sobre un usuario x perteneciente al conjunto de usuarios del sistema, con rango {verdadero, falso}. El dominio de x es el conjunto de todos los posibles perfiles de artistas digitales.

Predicado	Descripción	Tipo / Rango
tiene_ipad	El usuario tiene un iPad	Unario / {V,F}
tiene_tablet_android	El usuario tiene tablet Android con lapiz	Unario / {V,F}
tiene_pc_windows	El usuario trabaja en PC/laptop Windows	Unario / {V,F}
tiene_mac	El usuario trabaja en Mac	Unario / {V,F}
tiene_tableta_grafica	Tiene tableta grafica (Wacom u otra)	Unario / {V,F}
es_principiante	Sin experiencia previa en arte digital	Unario / {V,F}
es_intermedio	Experiencia basica-media en arte digital	Unario / {V,F}
es_profesional	Trabaja o estudia arte/diseno profesional	Unario / {V,F}
hace_ilustracion	Quiere hacer ilustracion digital	Unario / {V,F}
hace_vectorial	Necesita graficos vectoriales	Unario / {V,F}
hace_concept_art	Quiere hacer concept art	Unario / {V,F}
hace_manga_comic	Quiere hacer manga, comic o novela grafica	Unario / {V,F}

hace_uiux	Trabaja en diseno de interfaces UI/UX	Unario / {V,F}
hace_animacion	Quiere crear animaciones o GIFs	Unario / {V,F}
presupuesto_bajo	Prefiere herramientas gratuitas o pago unico	Unario / {V,F}
requiere_colaboracion	Necesita colaborar en tiempo real	Unario / {V,F}

3.2 Predicados de conclusión

Los siguientes predicados son derivados exclusivamente por el motor de inferencia y no pueden ser ingresados como hechos iniciales:

- **recomendar_procreate(x)**: El sistema recomienda Procreate para el usuario x
- **recomendar_clip_studio(x)**: El sistema recomienda Clip Studio Paint para el usuario x
- **recomendar_krita(x)**: El sistema recomienda Krita para el usuario x
- **recomendar_photoshop(x)**: El sistema recomienda Adobe Photoshop para el usuario x
- **recomendar_illustrator(x)**: El sistema recomienda Adobe Illustrator para el usuario x
- **recomendar_inkscape(x)**: El sistema recomienda Inkscape para el usuario x
- **recomendar_affinity(x)**: El sistema recomienda Affinity Designer para el usuario x
- **recomendar_figma(x)**: El sistema recomienda Figma para el usuario x
- **recomendar_sketchbook(x)**: El sistema recomienda Autodesk Sketchbook para el usuario x

3.3 Reglas en logica de predicados

A continuacion se presentan las 15 reglas formalizadas. El cuantificador universal $\forall x$ indica que la regla aplica a cualquier usuario x que satisfaga las condiciones.

- R1: $\forall x [\text{tiene_ipad}(x) \wedge \text{hace_ilustracion}(x)] \rightarrow \text{recomendar_procreate}(x)$
- R2: $\forall x [\text{tiene_ipad}(x) \wedge \text{hace_manga_comic}(x)] \rightarrow \text{recomendar_clip_studio}(x)$
- R3: $\forall x [\text{tiene_ipad}(x) \wedge \text{hace_concept_art}(x)] \rightarrow \text{recomendar_procreate}(x)$
- R4: $\forall x [\text{tiene_pc_windows}(x) \wedge \text{presupuesto_bajo}(x) \wedge \text{hace_ilustracion}(x)] \rightarrow \text{recomendar_krita}(x)$
- R5: $\forall x [\text{tiene_pc_windows}(x) \wedge \text{tiene_tableta_grafica}(x) \wedge \text{es_profesional}(x)] \rightarrow \text{recomendar_photoshop}(x)$
- R6: $\forall x [\text{tiene_pc_windows}(x) \wedge \text{hace_manga_comic}(x)] \rightarrow \text{recomendar_clip_studio}(x)$
- R7: $\forall x [\text{tiene_pc_windows}(x) \wedge \text{hace_animacion}(x) \wedge \text{presupuesto_bajo}(x)] \rightarrow \text{recomendar_krita}(x)$
- R8: $\forall x [\text{hace_vectorial}(x) \wedge \text{es_profesional}(x)] \rightarrow \text{recomendar_illustrator}(x)$
- R9: $\forall x [\text{hace_vectorial}(x) \wedge \text{presupuesto_bajo}(x)] \rightarrow \text{recomendar_inkscape}(x)$
- R10: $\forall x [\text{hace_vectorial}(x) \wedge \text{es_profesional}(x) \wedge \neg \text{presupuesto_bajo}(x)] \rightarrow \text{recomendar_affinity}(x)$
- R11: $\forall x [\text{hace_uiux}(x)] \rightarrow \text{recomendar_figma}(x)$
- R12: $\forall x [\text{requiere_colaboracion}(x) \wedge \text{hace_uiux}(x)] \rightarrow \text{recomendar_figma}(x)$
- R13: $\forall x [\text{tiene_mac}(x) \wedge \text{hace_vectorial}(x) \wedge \text{es_profesional}(x)] \rightarrow \text{recomendar_affinity}(x)$
- R14: $\forall x [\text{es_principiante}(x) \wedge \text{tiene_pc_windows}(x) \wedge \text{hace_ilustracion}(x)] \rightarrow \text{recomendar_krita}(x)$

R15: $Ax [es_principiante(x) \wedge tiene_tablet_android(x)] \rightarrow recomendar_sketchbook(x)$

3.4 Reglas en formato SI-ENTONCES con factores de certeza

El Factor de Certeza (FC) indica la confianza del experto en cada regla, en una escala de 0.0 a 1.0. Un FC de 0.95 significa que la regla es correcta en la gran mayoría de casos, admitiendo excepciones marginales.

Regla	Condiciones (SI)	Conclusion (ENTONCES)	FC
R1	tiene_ipad AND hace_ilustracion	recomendar_procreate	0.95
R2	tiene_ipad AND hace_manga_comic	recomendar_clip_studio	0.92
R3	tiene_ipad AND hace_concept_art	recomendar_procreate	0.90
R4	tiene_pc_windows AND presupuesto_bajo AND hace_ilustracion	recomendar_krita	0.93
R5	tiene_pc_windows AND tiene_tableta_grafica AND es_profesional	recomendar_photoshop	0.88
R6	tiene_pc_windows AND hace_manga_comic	recomendar_clip_studio	0.94
R7	tiene_pc_windows AND hace_animacion AND presupuesto_bajo	recomendar_krita	0.80
R8	hace_vectorial AND es_profesional	recomendar_illustrator	0.95
R9	hace_vectorial AND presupuesto_bajo	recomendar_inkscape	0.88
R10	hace_vectorial AND es_profesional AND NOT presupuesto_bajo	recomendar_affinity	0.85
R11	hace_uiux	recomendar_figma	0.97
R12	requiere_colaboracion AND hace_uiux	recomendar_figma	0.97
R13	tiene_mac AND hace_vectorial AND es_profesional	recomendar_affinity	0.87
R14	es_principiante AND tiene_pc_windows AND hace_ilustracion	recomendar_krita	0.91
R15	es_principiante AND tiene_tablet_android	recomendar_sketchbook	0.85

4. Motor de inferencia

4.1 Algoritmo de encadenamiento hacia adelante

El motor implementa encadenamiento hacia adelante basado en la regla de inferencia Modus Ponens: si se conoce que P es verdadero y existe la regla $P \rightarrow Q$, entonces se puede concluir Q.

El algoritmo opera de la siguiente manera:

8. Inicializar la memoria de trabajo con los hechos iniciales del usuario.
9. Establecer una bandera nuevos_hechos = False para controlar la iteracion.
10. Recorrer todas las reglas de la base de conocimiento.
11. Para cada regla, verificar si TODAS sus condiciones estan satisfechas en la memoria de trabajo (funcion_todas_condiciones_satisfechas).
12. Si la regla se puede disparar y su conclusion aun no esta en memoria: agregar la conclusion, registrar la regla en el historial y activar nuevos_hechos = True.
13. Al terminar el ciclo: si nuevos_hechos == False, se alcanzo el punto fijo y el proceso termina. Si True, iniciar un nuevo ciclo desde el paso 3.
14. Retornar la memoria de trabajo final y el historial de reglas aplicadas.

Se implementa un límite de 100 iteraciones como salvaguarda contra posibles ciclos infinitos, aunque la naturaleza de las reglas actuales hace que esto no ocurra en la práctica.

4.2 Evaluación de condiciones con negación

Cada condición es una tupla (predicado, valor_esperado). Esto permite representar tanto condiciones positivas como negaciones:

- ("hace_vectorial", True): el hecho debe estar presente y ser True
- ("presupuesto_bajo", False): el hecho debe ser False o estar ausente

Esta representación corresponde directamente al operador lógico NOT (-) en las reglas de lógica de predicados, sin necesidad de bibliotecas externas.

4.3 Sistema de explicación

El motor incluye dos mecanismos de explicación:

explicar_razonamiento()

Genera una descripción completa de la cadena de inferencia: muestra cada regla disparada en orden, sus condiciones verificadas, la conclusión derivada, el factor de certeza y la justificación en lenguaje natural.

porque(conclusion_buscada)

Función de explicación focalizada: dado el nombre de una conclusión específica, busca en el historial la regla que la origino y explica detalladamente por qué se disparó esa regla con los hechos del usuario.

4.4 Manejo de casos sin conclusión

Cuando ningun hecho inicial satisface las condiciones de ninguna regla, el motor retorna la memoria de trabajo sin cambios y el historial vacio. La funcion obtener_recomendaciones() detecta esta situacion y la interfaz muestra un mensaje claro indicando que no fue posible generar una recomendacion, sugiriendo al usuario agregar mas detalles de su perfil.

5. Implementación técnica

5.1 Tecnologías utilizadas

Tecnologia	Version	Uso en el proyecto
Python	3.8+	Lenguaje principal del backend
Streamlit	>=1.28.0	Framework para la interfaz web
Bibliotecas estandar	Python std	No se requieren dependencias externas para el motor

5.2 Estructura de archivos

- src/base_conocimiento.py: Diccionarios PREDICADOS, lista REGLAS y mapeo NOMBRES_HERRAMIENTAS
- src/motor_inferencia.py: Funciones de encadenamiento e inferencia
- src/app.py: Interfaz web con Streamlit
- tests/casos_prueba.md: Documentacion de los 5 casos de prueba
- README.md: Instrucciones de instalacion y ejecucion
- requirements.txt: Dependencias del proyecto

5.3 Decisiones de diseño

Representación de reglas como diccionarios: Se eligió una estructura de diccionarios Python en lugar de objetos o clases para mantener el código accesible y comprensible, facilitando la sustentación y modificación en tiempo real.

Session state de Streamlit: Los resultados de la inferencia se almacenan en st.session_state para que persistan entre interacciones del usuario (cambio de pestanas, consulta de 'Por que?') sin re-ejecutar el motor.

6. Validación

6.1 Tabla de casos de prueba

Caso	Descripcion	Hechos iniciales clave	Reglas activadas	Conclusion esperada	Estado
1	Ilustrador con iPad	tiene_ipad, hace_ilustracion	R1	Procreate	✓ OK
2	Diseñador UI/UX profesional con colaboración	hace_uiux, es_profesional, requiere_colaboracion	R11, R12	Figma	✓ OK
3	Principiante en PC sin presupuesto	tiene_pc_windows, es_principiante, hace_ilustracion, presupuesto_bajo	R4, R14	Krita	✓ OK
4	Artista de manga en PC con tableta	tiene_pc_windows, hace_manga_comic, tiene_tableta_grafica	R6	Clip Studio	✓ OK
5	Caso borde sin recomendación	tiene_tablet_android, hace_animacion	ninguna	Sin resultado	✓ OK

6.2 Análisis de resultados

Los cinco casos de prueba validaron correctamente el comportamiento del sistema:

- Casos 1 a 4 confirmaron que el motor deriva las conclusiones correctas para perfiles típicos de usuarios reales.
- El caso 2 verifico que múltiples reglas pueden dispararse para la misma conclusión (R11 y R12) sin duplicar el resultado, gracias a la verificación de hechos ya presentes en la memoria.
- El caso 3 valido que dos reglas distintas (R4 y R14) pueden converger en la misma recomendación (Krita) de forma consistente.
- El caso 5 (caso borde) confirmo que el sistema maneja correctamente la ausencia de recomendaciones sin errores ni comportamientos inesperados.

6.3 Limitaciones identificadas

- Mundo cerrado: el sistema asume que todo hecho no declarado es falso. Perfiles parciales pueden no generar recomendaciones, aunque exista una válida.
- Sin resolución de conflictos: cuando dos recomendaciones distintas son igualmente válidas, el sistema las presenta todas sin priorizar.
- Predicados binarios únicamente: no captura gradaciones (por ejemplo, nivel de experiencia intermedio-avanzado o presupuesto medio).
- Dominio acotado: no considera herramientas emergentes ni actualizaciones de precios o disponibilidad.

7. Conclusiones

7.1 Logros alcanzados

- Se implemento un sistema experto funcional con 15 reglas de producción y 16 predicados, capaz de recomendar entre 9 herramientas de dibujo digital.
- El motor de inferencia con encadenamiento hacia adelante deriva correctamente todas las conclusiones posibles a partir de los hechos iniciales.
- La interfaz web en Streamlit permite a usuarios no técnicos consultar el sistema de forma intuitiva, con visualización clara de resultados y explicaciones detalladas.
- Los 5 casos de prueba documentados pasaron correctamente, incluyendo el caso borde sin recomendación.

7.2 Aprendizajes obtenidos

- La formalización en lógica de predicados obliga a ser explícito sobre cada condición, eliminando la ambigüedad del conocimiento experto informal.
- El encadenamiento hacia adelante es eficiente para dominios donde los hechos iniciales son conocidos y se busca derivar todas las conclusiones posibles.
- La separación entre base de conocimiento y motor de inferencia facilita enormemente el mantenimiento y la expansión del sistema.
- La representación de negaciones como tuplas (predicado, False) demuestra que problemas aparentemente complejos tienen soluciones simples y elegantes en Python.

7.3 Posibles mejoras futuras

- Agregar predicados con valores continuos (nivel de experiencia como numero, presupuesto en rango) usando lógica difusa.
- Ampliar la base de conocimiento con herramientas adicionales: Clip Studio para iOS, Procreate Dreams para animación, Adobe Fresco.
- Incorporar un módulo de aprendizaje que ajuste los factores de certeza según el feedback de los usuarios.